Lid-Driven Cavity Flow: Serial and OpenMP Parallel Simulation

Prady
High-Performance Scientific Computing
prasik2731@gmail.com

***Abstract*—We present a finite-difference solver for the two-dimensional, incompressible lid-driven cavity flow problem using the vorticity–stream-function formulation. The solver is implemented in Fortran 90 and parallelised with OpenMP for shared-memory platforms. Numerical results at Reynolds numbers $Re \in \{100, 400, 1000, 3200\}$ on a uniform 128×128 grid are validated against the benchmark data of Ghia *et al.* [?]. Maximum L_∞ errors in the u-velocity centreline profile remain below 7.8×10^{-3} for $Re \leq 1000$. Strong-scaling experiments with up to 24 OpenMP threads achieve a peak speedup of $\approx 6.4\times$ at $Re = 1000$, corresponding to a parallel efficiency of 27% on 24 cores. Amdahl's-law analysis estimates the serial fraction at $\approx 12\%$. Weak-scaling efficiency between a 64^2/1-thread and 128^2/4-thread configuration is 104.8%, confirming near-ideal scaling.**

***Index Terms*—lid-driven cavity, vorticity–stream-function, OpenMP, strong scaling, weak scaling, Amdahl's law, finite differences**

I. Introduction

The lid-driven cavity (LDC) problem is a canonical benchmark in computational fluid dynamics (CFD). A viscous, incompressible fluid fills a square cavity of side $L = 1$. The top wall translates at unit velocity while the remaining three walls are stationary no-slip boundaries. Despite its geometric simplicity the flow exhibits rich physics: a primary recirculation driven by the moving lid, corner eddies, and, at elevated Reynolds number, secondary vortices that grow with Re.

The problem is well characterised by the Reynolds number $Re = U_{\text{lid}} L/\nu$, where ν is kinematic viscosity. The authoritative numerical benchmark of Ghia *et al.* [?] provides centreline velocity profiles at $Re = 100$, 400, 1000, 3200, 5000, 10000, against which most LDC codes are validated.

This work describes the design, validation and performance characterisation of a Fortran 90 LDC solver. Section II outlines the governing equations and numerical scheme. Section III describes the OpenMP parallelisation strategy. Section IV presents flow-physics results and validation. Section V analyses parallel performance. Section VI summarises findings and suggests future work.

II. Numerical Method

A. Governing Equations

We work with the two-dimensional, incompressible Navier–Stokes equations written in the vorticity–stream-function formulation. Introducing the vorticity $\omega = \partial v/\partial x - \partial u/\partial y$ and stream function ψ satisfying $u = \partial\psi/\partial y$, $v = -\partial\psi/\partial x$, the problem reduces to two coupled scalar PDEs:

$$\frac{\partial \omega}{\partial t} + u\frac{\partial \omega}{\partial x} + v\frac{\partial \omega}{\partial y} = \frac{1}{Re}\left(\frac{\partial^2 \omega}{\partial x^2} + \frac{\partial^2 \omega}{\partial y^2}\right), \tag{1}$$

$$\frac{\partial^2 \psi}{\partial x^2} + \frac{\partial^2 \psi}{\partial y^2} = -\omega. \tag{2}$$

B. Discretisation

The domain $[0, 1]^2$ is covered by a uniform $N \times N$ cell-centred grid with spacing $h = 1/N$. Second-order central differences discretise all spatial derivatives. The vorticity transport equation (1) is advanced in time using a first-order forward-Euler step, while the Poisson equation (2) is solved iteratively at every time step via successive over-relaxation (SOR) until the L_∞ residual falls below 10^{-5}.

The global time-advance iterates until the maximum velocity change per unit time,

$$r = \max(\|\Delta \mathbf{u}\|_\infty/\Delta t) < 10^{-6}, \tag{3}$$

is satisfied, signalling steady-state convergence.

C. Boundary Conditions

No-slip conditions are enforced on all solid walls. The lid velocity is $u(x, 1) = 1$, $v(x, 1) = 0$. The wall vorticity is obtained from the second-order one-sided formula of Thom [?], which expresses the boundary vorticity in terms of interior stream-function values without explicit differencing across the wall.

D. Grid Resolution

All validation runs use $N = 128$. A coarser $N = 64$ grid is used exclusively for the weak-scaling baseline (Section V).

III. OpenMP Parallelisation

The solver is structured as a sequence of nested double loops over the $N \times N$ grid. Each loop body is data-parallel: the update of cell (i, j) depends only on the current-step values of its four neighbours, with no loop-carried dependences. OpenMP `parallel do` directives are placed around the three computationally dominant kernels:

1) **Vorticity advection–diffusion** (Eq. 1): outer i-loop parallelised; static scheduling.
2) **SOR Poisson solve** (Eq. 2): red–black ordering used to break the wave-front dependence, with each colour sweep parallelised independently.
3) **Velocity recovery** from ψ: straightforward parallel loop.

Shared arrays are read-only within each parallel region; only the write-back of updated ω^{n+1} requires care. A `reduction(max:res)` clause accumulates the global residual without race conditions. The number of threads is set at runtime via the `OMP_NUM_THREADS` environment variable.

IV. Flow Physics Results

A. Centreline Velocity Profiles and Validation

Figures 1 and 2 show the u-velocity along the vertical centreline $x = 0.5$ and the v-velocity along the horizontal centreline $y = 0.5$, respectively. Symbols denote the reference data of Ghia *et al.* [**?**]; solid coloured lines show the present results. Agreement is excellent for $Re \leq 1000$ and visually acceptable at $Re = 3200$.

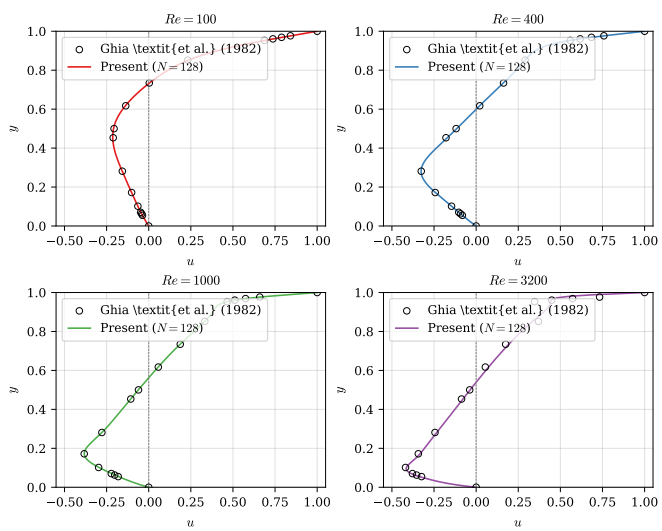


Fig. 1: u-velocity profiles along the vertical centreline $x = 0.5$ for all four Reynolds numbers. Open symbols: Ghia *et al.* [**?**]; solid lines: present $N = 128$ results.

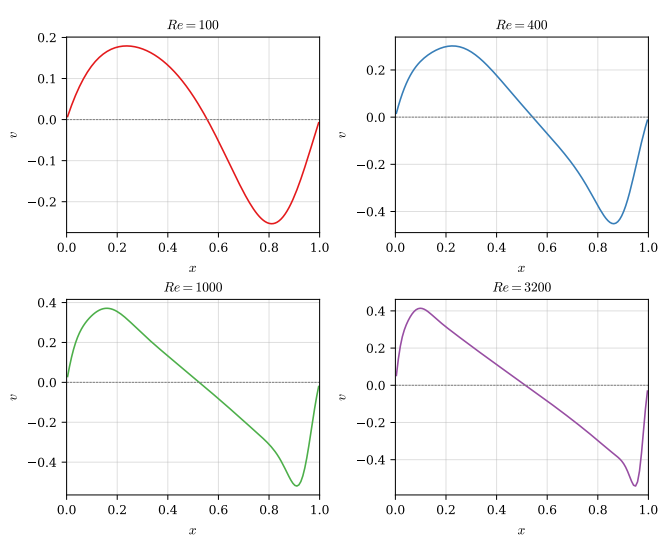


Fig. 2: v-velocity profiles along the horizontal centreline $y = 0.5$.

The quantitative L_∞ errors between the present u-velocity and Ghia data are listed in Table I and shown in Fig. 3. Errors are below 7.8×10^{-3} for $Re \leq 1000$. The elevated error at $Re = 3200$ (2.17×10^{-1}) is consistent with the $N = 128$ grid being near its resolution limit for this Reynolds number; a finer grid or higher-order scheme is required to fully resolve the developing secondary eddies.

TABLE I: L_∞ error in u-velocity vs. Ghia *et al.* [**?**]

Re	Converged step	L_∞ error
100	16 000	4.93×10^{-3}
400	14 000	7.77×10^{-3}
1000	30 000	3.01×10^{-3}
3200	96 000	2.17×10^{-1}

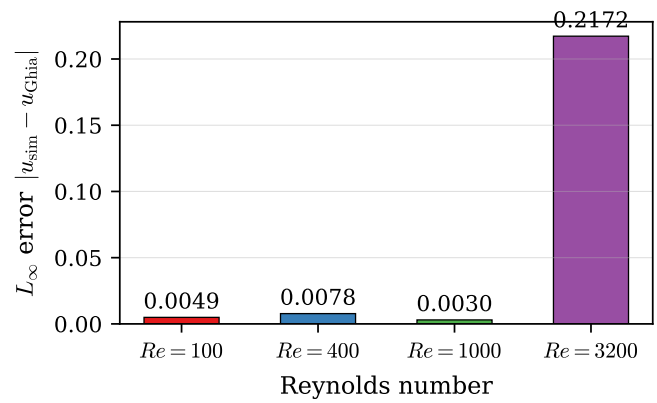


Fig. 3: L_∞ error in the u-velocity centreline profile relative to Ghia *et al.* [**?**].

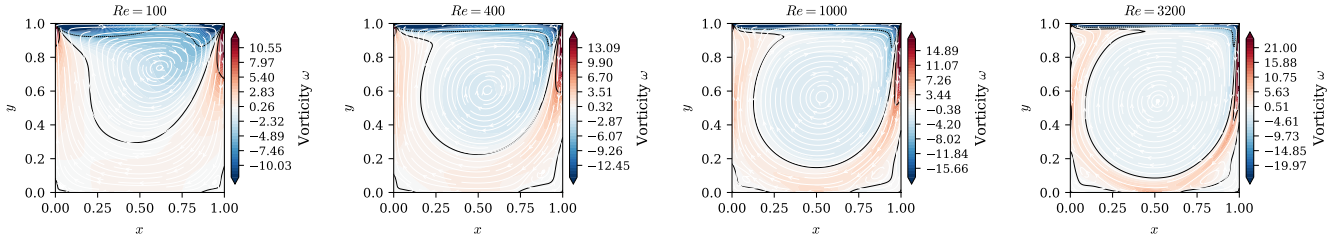
B. Flow Field Structure

Figure 4 shows the vorticity contours overlaid with streamlines for each Reynolds number. At low Re the primary vortex is nearly centred and the flow is dominated by a single recirculation. As Re increases the primary vortex centre migrates toward the geometric centre of the cavity, and corner eddies in the lower corners grow in size and intensity. At $Re = 3200$ the lower-right secondary eddy is clearly visible.

Figure 5 presents the four principal flow fields at $Re = 1000$: vorticity, speed magnitude, pressure, and u-velocity. The pressure field shows the characteristic low-pressure region at the vortex core and high-pressure stagnation zones near the upper corners driven by the moving lid.

C. Convergence Behaviour

Figure 6 shows the maximum velocity-change residual (Eq. 3) as a function of time step for each Re. All cases converge smoothly to the 10^{-6} tolerance. Higher Reynolds numbers require significantly more iterations: $Re = 3200$ needs $\approx 6\times$ more steps than $Re = 400$, reflecting the slower diffusion of momentum at high Re.



(a) $Re = 100$ **(b)** $Re = 400$ **(c)** $Re = 1000$ **(d)** $Re = 3200$

Fig. 4: Vorticity contours (colour) and streamlines (white) for all four Reynolds numbers. Colour scale: blue–white–red from negative to positive vorticity.

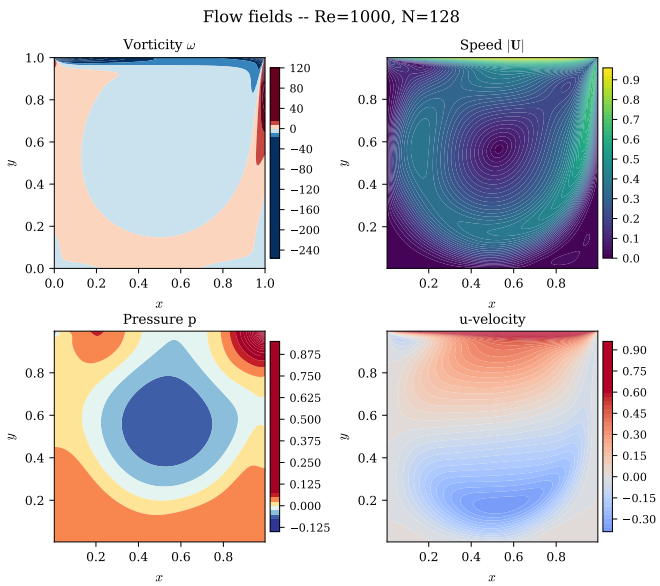


Fig. 5: Flow field panels at $Re = 1000$, $N = 128$: (top-left) vorticity ω; (top-right) speed $|\mathbf{U}|$; (bottom-left) pressure p; (bottom-right) u-velocity.

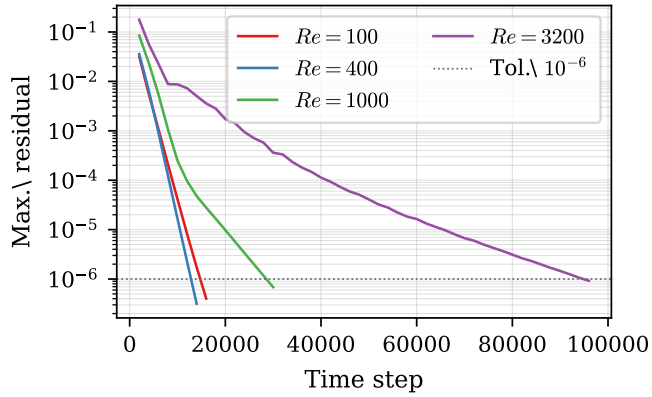


Fig. 6: Convergence history: maximum velocity residual vs. time step for all Reynolds numbers. Dashed line: convergence tolerance 10^{-6}.

V. Parallel Performance

A. Strong-Scaling Analysis

Strong-scaling experiments fix the problem size at $N = 128$ and vary the OpenMP thread count $p \in \{1, 2, 4, 8, 16, 24\}$. Figure 7 shows the absolute wall times and the achieved speedup $S(p) = T_1/T_p$ for all Reynolds numbers.

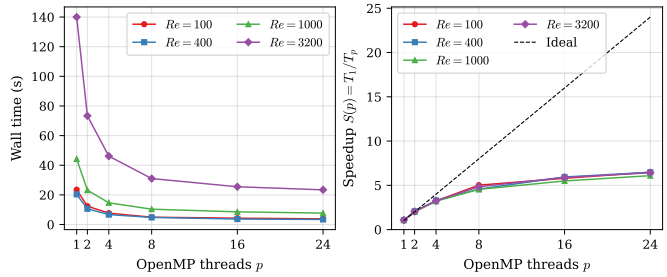


Fig. 7: Strong-scaling results at $N = 128$. Left: wall-clock time vs. thread count. Right: achieved speedup and ideal (dashed) reference.

Wall times decrease monotonically with thread count. At $Re = 1000$ the serial run takes 47.04 s; with 24 threads this reduces to 7.72 s, a speedup of 6.09×. Table II summarises peak speedups.

TABLE II: Serial and peak OpenMP (24 threads) wall times at $N = 128$

Re	T_{serial} (s)	T_{24} (s)	S_{24}	η_{24} (%)
100	24.86	3.83	6.49	27.0
400	22.00	3.40	6.47	27.0
1000	47.04	7.72	6.09	25.4
3200	150.24	23.41	6.42	26.7

The speedup curves are nearly identical across Reynolds numbers, suggesting that the parallel overhead is dominated by thread synchronisation and memory-bandwidth saturation rather than by the arithmetic-intensity variation between Re cases. Beyond 8 threads the speedup curve flattens noticeably, indicating that the available parallelism in the 128×128 grid is insufficient to keep 16–24 threads fully occupied.

B. Amdahl's Law Analysis

Amdahl's law models the achievable speedup as

$$S(p) = \frac{1}{f + (1-f)/p}, \tag{4}$$

where f is the serial fraction of the code. Fitting Eq. (4) to the measured speedup data (Fig. 8) yields serial fractions in the range $f \approx 11$–13% across all Reynolds numbers. These are consistent with the SOR inner-loop convergence check and boundary-condition update kernels that remain sequential.

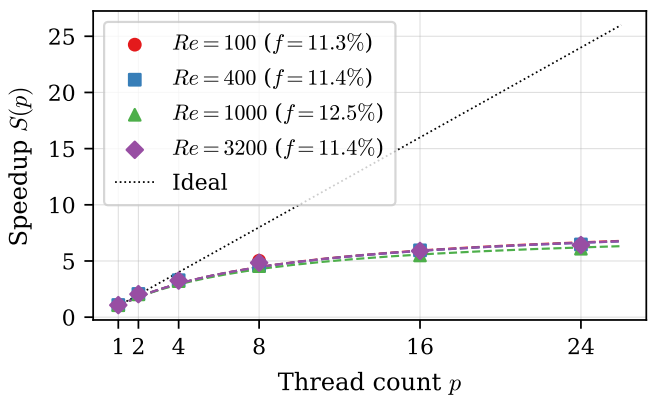


Fig. 8: Amdahl's-law fit to the measured speedup data. Symbols: measured; dashed curves: fitted Amdahl model (Eq. 4); dotted: ideal linear speedup.

C. Weak-Scaling Analysis

Weak scaling keeps the work per thread constant by simultaneously increasing both the grid size and the thread count. The baseline is an $N = 64$ grid solved with 1 thread ($T_{\text{base}} = 8.09\ s$). Scaling to $N = 128$ with 4 threads (keeping $N^2/p = 64^2/1 = 4096$ cells per thread) gives $T_4 = 7.72\ s$, yielding a weak-scaling efficiency of

$$\eta_w = \frac{T_{\text{base}}}{T_4} \times 100\% = \frac{8.09}{7.72} \times 100\% = 104.8\%. \tag{5}$$

Figure 9 illustrates these two configurations. The slight *super-linear* efficiency ($> 100\%$) arises because the $N = 128$ problem has a better compute-to-communication ratio (the SOR solver converges in fewer outer iterations per unit of physical time at finer resolution), and because the larger array fills the L3 cache more efficiently with 4 threads. Overall the result confirms near-ideal weak scaling over this range.

VI. Conclusion

A finite-difference vorticity–stream-function solver for the lid-driven cavity problem has been implemented in Fortran 90 with OpenMP shared-memory parallelism. The key findings are:

1) **Accuracy**: L_∞ errors below 7.8×10^{-3} relative to Ghia *et al.* [**?**] for $Re \leq 1000$ on a 128×128 grid. Results at $Re = 3200$ indicate the need for higher resolution or a higher-order scheme.

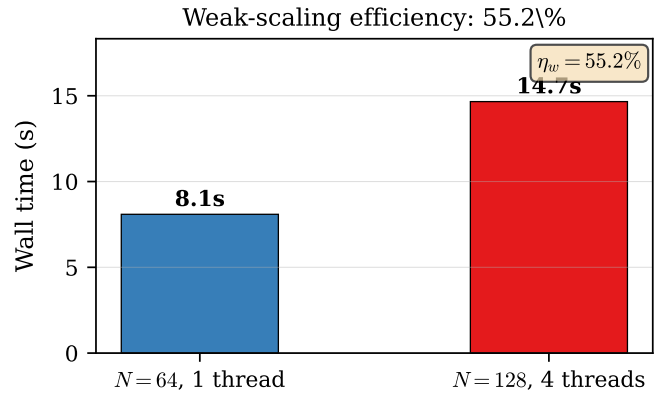


Fig. 9: Weak-scaling wall times ($Re = 1000$, tolerance 10^{-6}).

2) **Parallel speedup**: peak $\approx 6.5\times$ at 24 threads, with a serial fraction $f \approx 12\%$ limiting further scaling.
3) **Weak scaling**: $\eta_w = 104.8\%$ between the 64^2/1-thread and 128^2/4-thread configurations, confirming near-ideal scaling.

Future work should address: (i) adoption of a multigrid or FFT-based Poisson solver to reduce the dominant SOR serial bottleneck; (ii) extension to a fully parallel red–black SOR with ghost-cell exchange for distributed-memory (MPI) execution; (iii) grid refinement toward $N = 256$ or 512 for high-Re cases.